CareTrack - Emergency Email Notifications

How fall detection and other emergencies notify caregivers by email | Backend v2.0.0 | Generated 2026-10-14

1. Overview

CareTrack evaluates every incoming telemetry event (wearable cardiac data and room-unit data) for emergencies. When a fall or a dangerous vital sign is detected, the backend records an **Alert**, emails every caregiver responsible for that patient, and records one **Notification** row per caregiver so the delivery outcome is auditable from the dashboard.

- Falls are always **CRITICAL** and always produce an email.
- Vital-sign breaches (heart rate, SpO_2, body temperature) are compared against caregiver-configurable thresholds.
- Room-temperature breaches produce an **ENVIRONMENT / WARNING** email.
- All emergencies found on one telemetry event are batched into a **single email** (one message per incident, not per metric).
- A per-device cooldown prevents a burst of telemetry from flooding inboxes.
- The pipeline is **inert until SMTP_HOST is configured**: alerts and notifications are still recorded (marked FAILED), but no mail is sent.

2. End-to-end pipeline

```
telemetry event   MQTT  caretrack/room/data, caretrack/cardiac/data
                  or    POST /api/v1/telemetry/ingest
     |
     v
parse_telemetry()              -> TelemetryEvent          telemetry.py
     |
     v
alert_worker(event)            thread-pool worker         main.py:78
  (never blocks the MQTT loop or the HTTP webhook)
     |
     v
process_emergency(event)       single entry point         core/alerting.py
  |-- detect_emergencies()     event vs thresholds
  |-- cooldown check           (device, category, metric)
  |-- persist Alert row(s)     status = active
  |-- resolve_recipients()     patient -> caregivers
  |-- send_alert_email()       one SMTP send per event    core/email.py
  `-- persist Notification(s)  DELIVERED / FAILED + error
```

3. Detection rules

Detection lives in `detect_emergencies()` (core/alerting.py). Thresholds marked *config* come from `ACTIVE_THRESHOLDS` (routers/thresholds.py) and can be changed at runtime via `GET/PUT /api/v1/thresholds`. Critical cut-offs are fixed in code.

Metric	Triggers when	Category	Severity	Defaults / notes
fall_detected	reading is true	fall	CRITICAL (always)	Impact in g reported from accel_magnitude.
heart_rate	value < hr_min or > hr_max	cardiac	WARNING	hr_min=50, hr_max=110 (config).
heart_rate	value < 40 or > 150	cardiac	CRITICAL	Physiologically dangerous range (fixed).
spo2	value < spo2_min	cardiac	CRITICAL	spo2_min=92 (config).
body_temp	value <= 35.0 or >= 39.0 C	cardiac	CRITICAL	Hypothermia / febrile cut-offs (fixed).
room_temp	value < 18.0 or > 28.0 C	environment	WARNING	room_temp_min/max (config).

Events that trigger nothing return from detection **before any database or network work happens**, so ordinary telemetry costs almost nothing.

4. Who receives the email

Recipients are resolved per event in `resolve_recipients()`:

- **Primary path:** device identifier -> Device -> latest **active** DevicePairing -> patient -> every **active** caregiver assigned to that patient (CaregiverAssignment).
- **Fallback:** if the device is unpaired or no assignments exist, every active CAREGIVER and ADMIN user is emailed, so an unpaired device still raises a signal.
- **Always appended:** addresses in `ALERT_FALLBACK_RECIPIENTS` (e.g. a shared ward mailbox). They receive the email but get no Notification row, because they are not registered users.

5. What is written to the database

Table	Fields written	Purpose
alerts	patient_id (nullable), device_id, category, severity, triggering_metric, triggering_value, message, status=active	The incident record shown in the dashboard and worked to resolution.
notifications	alert_id, user_id, channel=email, status, delivered_at (UTC, only when delivered), error_message	One row per alert x registered recipient; the audit trail of the email attempt.

Notification status semantics:

Status	Set when	error_message
delivered	SMTP accepted the message.	(empty)
failed	SMTP_HOST is empty (pipeline inert).	SMTP not configured (SMTP_HOST is empty)
failed	EMAIL_ALERTS_ENABLED=false.	Email alerts disabled (EMAIL_ALERTS_ENABLED=false)
failed	No recipients could be resolved.	No recipients configured
failed	SMTP connection/login/send error.	ExceptionName: <message>

6. The email itself

Subject: `[CareTrack <SEVERITY>] <N> emergency alert(s) - <device_id>` (highest severity wins, pluralised when several emergencies batch). Plain-text body:

```
CareTrack emergency notification

Device     : ESP32-CardiacMonitor
Detected   : 2026-10-14T09:32:11.482000+00:00
Patient    : Margaret Thatcher (Room 204)

Detected emergencies:
  - [CRITICAL] Fall detected on device ESP32-CardiacMonitor (impact 3.85 g).
    Immediate assistance may be required.
  - [CRITICAL] Heart rate 162 bpm is outside the configured range 50-110 bpm.

Open the CareTrack dashboard to acknowledge and resolve this alert.
```

7. Cooldown (rate limiting)

- Key: `(device_id, category, metric)` - e.g. a sustained fall keeps re-triggering, but only the first event emails.
- Default gap: `ALERT_COOLDOWN_SECONDS=300` (5 minutes).
- The cooldown is stamped **before** sending, so the MQTT worker and the HTTP ingest worker cannot double-emit the same emergency.

- Suppressed repeats are logged: `[ALERT] fall/fall_detected on room-unit-01 suppressed (cooldown 271s left)`.
- Tests call `reset_cooldowns()` to clear the in-memory state.

8. Configuration (environment variables)

Set these in `backend/.env`; defaults shown. See the emergency block in `.env.example`.

Variable	Default	Meaning
EMAIL_ALERTS_ENABLED	true	Master switch; false short-circuits with a FAILED reason.
SMTP_HOST	(empty)	SMTP server. Empty keeps the pipeline inert (recommended until real SMTP is available).
SMTP_PORT	587	SMTP port (587 STARTTLS, 465 SSL, 25 plain).
SMTP_USERNAME	(empty)	Login user; login is skipped when empty.
SMTP_PASSWORD	(empty)	Login password.
SMTP_FROM	CareTrack Alerts <alerts@caretrack.local>	From header.
SMTP_USE_TLS	true	STARTTLS after connect.
SMTP_TIMEOUT_SECONDS	10	Socket timeout.
ALERT_COOLDOWN_SECONDS	300	Minimum gap per device/category/metric.
ALERT_FALLBACK_RECIPIENTS	(empty)	Extra comma-separated addresses always copied.

9. Where it is wired (backend/main.py)

- `alert_worker(event)` (main.py:78) wraps `process_emergency()` with error logging.
- **MQTT path:** the broker on_message handler parses the event, stores it, then schedules the worker via `main_loop.run_in_executor(None, alert_worker, event)` (main.py:143).
- **HTTP path:** `POST /api/v1/telemetry/ingest` (main.py:361) parses the payload, calls `store_event()`, then `run_in_executor(...)` (main.py:375) and responds immediately.
- Both paths share the same cooldown state, so an event seen twice (MQTT + HTTP relay) emails once.

10. Testing and simulating a fall

Run the test suite from `backend/`:

```
uv run pytest -q        # 24 passed (13 new in tests/test_alerting.py)
```

The new tests cover: fall is CRITICAL; normal vitals trigger nothing; configured thresholds; room-temperature warning; Alert + Notification persistence and delivery; SMTP failure marks FAILED; cooldown suppression; unpaired-device fallback; multi-breach batching; and the SMTP transport itself (message build, connection error, missing host, disable flag).

Simulate a fall against a running server (uv run uvicorn main:app --host 127.0.0.1 --port 8000):

```
curl -X POST http://127.0.0.1:8000/api/v1/telemetry/ingest \
  -H "Content-Type: application/json" \
  -d '{"device_id": "room-unit-01", "sensor": "wearable",
       "readings": {"fall_detected": true, "accel_magnitude": 3.85}}'
```

Expected: HTTP 200 with `"status": "success"`; one `alerts` row (fall / critical); one `notifications` row per registered caregiver. With SMTP_HOST empty the notification rows are FAILED with *SMTP not configured (SMTP_HOST is empty)*; set real SMTP values and repeat to see them flip to delivered. Wait ALERT_COOLDOWN_SECONDS (or restart the server) before re-testing the same device/metric.

11. File reference

File	Role
backend/core/alerting.py	Detection rules, recipient resolution, email rendering, cooldown, Alert/Notification persistence, process_emergency() entry point.
backend/core/email.py	SMTP transport (stdlib smtplib): send_smtp_mail(), send_alert_email(); returns (ok, detail), never raises.
backend/main.py	alert_worker() and the MQTT / HTTP ingest wiring.
backend/settings.py	SMTP + cooldown + fallback-recipient settings (env-backed).
backend/.env.example	Documented SMTP block to copy into backend/.env.
backend/routers/thresholds.py	ACTIVE_THRESHOLDS defaults and GET/PUT /api/v1/thresholds runtime updates.
backend/tests/test_alerting.py	13 tests for the whole pipeline.
backend/models/alert.py, notification.py	Alert and Notification ORM models.